



L'idée du "parcours" est de "visiter" tous les sommets d'un graphe en partant d'un sommet quelconque. Ces algorithmes de parcours d'un graphe sont à la base de nombreux algorithmes : recherche de cycles dans le graphe, recherche du chemin le plus court pour aller d'une ville à une autre, routage des paquets de données dans un réseau,...



Il existe 2 méthodes pour parcourir un graphe :

- le parcours en profondeur d'abord (*Depth First Search*)
- Le parcours en largeur d'abord (*Breadth First Search*)

Video Parcours en profondeur <https://www.youtube.com/embed/-wPLAsAuenA?feature=oembed>

Video Parcours en largeur <https://www.youtube.com/embed/8vnbENILU5E?feature=oembed>

I. PARCOURS EN PROFONDEUR

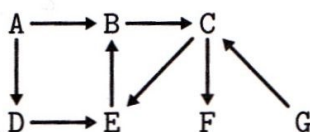
1. LE PRINCIPE

Parcourir un graphe en profondeur à partir d'un sommet particulier (la source), consiste à l'explorer de la manière suivante :

En partant de la source, on suit un chemin, en délaissant au passage certains autres, jusqu'à devoir rebrousser chemin lorsqu'on arrive sur un sommet qui n'a plus de voisins non encore visités. On coche au passage chaque sommet visité.

En rebroussant chemin, on explore les chemins délaissés à l'aller.

Ex 1 : Sur le graphe orienté suivant en démarrant du sommet 'A' :



Compléter les commentaires et la construction du parcours : [G,C,F,E,B]

sommets visités	commentaires	parcours (sommets cochés)
A	cocher A, emprunter arc $A \rightarrow B$	[A]
. B	cocher B, emprunter arc $B \rightarrow C$	[A,B]
. . C	cocher C, emprunter arc $C \rightarrow E$	[A,B,C]
. . . E	cocher E, emprunter arc $E \rightarrow B$	[A,B,C,E]
. . . . B	déjà vu, demi-tour...	
. . . E	pas d'autre arc, retourner encore sur ses pas	
. . C	emprunter arc $C \rightarrow F$	
. . . F	cocher F, sans issue, demi-tour...	[A,B,C,E,F]
. . C	pas d'autre arc, retourner encore sur ses pas	
. B	pas d'autre arc, retourner encore sur ses pas	
A		
. D		
. . E		
. D		
A		

Rq1 : Il y a dans l'ordre de ce parcours une part d'indéterminisme, puisque plusieurs choix sont possibles au départ d'un sommet de degré ≥ 2 .

Exo 1.

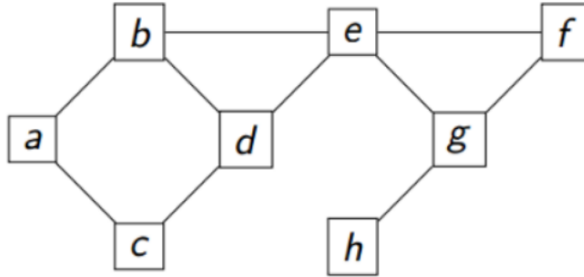
1. Donner la liste des sommets cochés dans un parcours en profondeur à partir de A en choisissant d'emprunter d'abord l'arc $A \rightarrow D$

2. Donner la liste des sommets cochés dans un parcours en profondeur à partir de C

Rq2. Le sommet G n'a jamais été coché : il n'est pas accessible à partir de A, pas plus qu'à partir de n'importe quel autre sommet du graphe

3. Donner la liste des sommets cochés dans un parcours en profondeur à partir de G

Exo 2. Donner la liste des sommets cochés dans un parcours en profondeur du graphe suivant non orienté en démarrant du sommet 'g' :



2. IMPLEMENTATION

a) Algorithme itératif



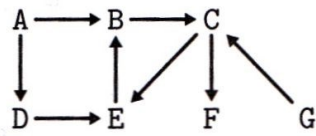
L'algorithme utilise une pile pour retenir les sommets à revisiter lorsque l'on revient sur ses pas.

fonction parcours_profondeur(G,sommet):

```

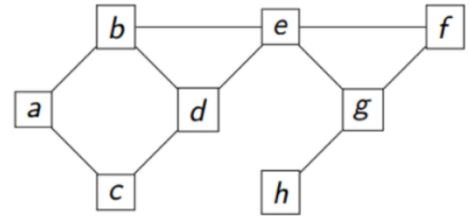
sommets_cochés ← liste vide
pile ← pile vide
On empile le sommet dans pile
Tant que pile n'est pas vide :
    s ← depile la pile
    si s n'est pas dans la liste sommets_cochés
        on ajoute s à la liste sommets_cochés
        voisins ← la liste des voisins de s non déjà cochés
        pour chaque voisin v de voisins (donc non déjà cochés)
            On empile v
    fin tant que
renvoyer sommets_cochés
    
```

Exo 3. Pour se convaincre de la conformité de cet algorithme, donner l'état des variables à l'issue de chaque itération, pendant l'exécution de `parcours_profondeur(G_1, 'A')` où **G_1** est le graphe déjà rencontré suivant :



	pile	s	voisins (non cochés)	sommets_cochés
initialisation	A			[]
itération 1			[D,B]	
itération 2				
itération 3			[F,E]	
itération 4				
itération 5				
itération 6				

Exo 4. Pour se convaincre encore davantage de la conformité de cet algorithme, donner l'état des variables à l'issue de chaque itération, pendant l'exécution de **parcours_profondeur(G_2,'g')** où **G_2** est le graphe déjà rencontré ci-contre :



	pile	s	voisins (non cochés)	sommets_cochés
initialisation				[]
itération 1				
itération 2				
itération 3				
itération 4				
itération 5				
itération 6				
itération 7				
itération 8				
itération 9				
itération 10				
itération 11				

Exo 5. En utilisant une classe **Pile** et une classe **Graphe** de votre choix, implémenter cet algorithme de parcours en profondeur. Tester sur les graphes **G_1** et **G_2**.



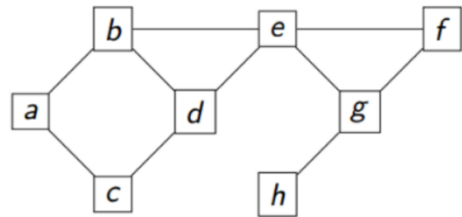
..\SRC\parcours_profondeur.py

b) Algorithme récursif



On peut utiliser un algorithme récursif pour parcourir un graphe en profondeur.

1. On part d'un sommet du graphe.
2. On le coche comme visité s'il ne l'est pas déjà.
3. Pour chacun de ses voisins non visités, on reprend à partir du 1.



Exo 6. En utilisant une classe **Graphe** de votre choix, implémenter cet algorithme de parcours en profondeur. Tester sur les graphes **G_1** et **G_2**.



..\SRC\parcours_profondeur.py

Rq 3. Dans le cas de la classe **Graphe** implémentée avec le dictionnaire d'adjacence, l'ordre des voisins n'est pas maîtrisé (c'est l'interpréteur *python* qui le détermine). L'ordre des sommets cochés est donc déterminé en partie par *Python*.

II. EXOS EN PROFONDEUR

Exo 7. Existence d'un chemin

Ecrire la fonction **existe_chemin(g,s,t)** qui utilise la fonction **parcours_profondeur()** et qui renvoie un booléen précisant s'il existe un chemin de **s** à **t** dans le graphe **g**.

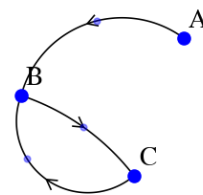


..\SRC\parcours_profondeur.py

Exo 8. Graphe Connexe

Un graphe est connexe si quels que soient les sommets **s** et **t**, il existe un chemin reliant **s** à **t**. (tous les sommets sont reliés entre eux par un chemin)

- a. On suppose que le graphe n'est pas orienté.
Ecrire la fonction **est_connexe_norienté(g)** qui utilise la fonction **parcours_profondeur()** et qui renvoie un booléen précisant si le graphe **g** est connexe ou non.
(Il n'est pas nécessaire de trouver tous les chemins reliant chaque couple de sommets)
- b. On suppose que le graphe peut être orienté ou non.
Ecrire la fonction **est_connexe(g)** qui utilise la fonction **parcours_profondeur()** et qui renvoie un booléen précisant si le graphe **g** est connexe ou non.



Le graphe ci-contre n'est pas connexe, puisqu'il n'y a pas de chemin de **C** à **A** :

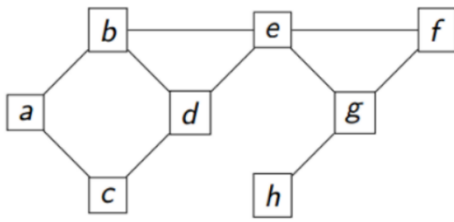


..\SRC\parcours_profondeur.py

Exo 9. Un chemin entre deux sommets

L'objectif est de faire afficher un chemin entre deux sommets d'un graphe.

Par exemple dans le graphe G₂ : a - b - e - g - h est l'un des chemins possibles entre a et h

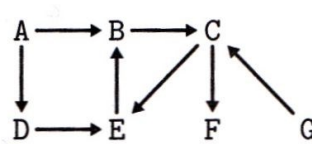


La méthode consiste à associer à chaque sommet dans un dictionnaire le sommet qui a permis de l'atteindre la première fois pendant le parcours en profondeur. (cette information est perdue dans le cas d'une simple liste des sommets parcourus, elle est ici nécessaire)

Le sommet de départ ne sera associé à rien (None)

Une fois le parcours en profondeur terminé, on cherche le sommet d'arrivée et on remonte le dictionnaire jusqu'au sommet de départ.

1. Ecrire une fonction récursive semblable à la fonction de parcours en profondeur récursive, qui renvoie le dictionnaire des arcs parcourus où la clé est l'extrémité de l'arc et la valeur est l'origine de l'arc.



`parcours_prof_rec(G_1, 'A')` renvoie `['A', 'B', 'C', 'E', 'F', 'D']`

`parcours_arcs(G_1, 'A')` renvoie `{'A': None, 'B': 'A', 'C': 'B', 'E': 'C', 'F': 'C', 'D': 'A'}`

2. Ecrire la fonction `chemin` qui prend en paramètre le sommet d'arrivée et le dictionnaire des arcs du parcours, et qui renvoie le chemin sous forme de liste des sommets qui le constituent.
3. Tester cette fonction sur les graphes G₁ et G₂. Le chemin produit est-il toujours le plus court ?

III. PARCOURS EN LARGEUR

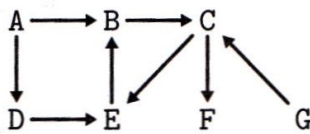
Le parcours en profondeur permet de détecter l'existence d'un chemin entre deux sommets mais il ne détermine pas pour autant la distance entre deux sommets c'est-à-dire la longueur minimale d'un chemin entre eux. En empruntant les arcs dans un ordre arbitraire (celui de l'interpréteur Python), le chemin produit n'est pas nécessairement de longueur minimale. Le parcours en largeur permet de produire ce chemin le plus court.

3. LE PRINCIPE

Parcourir un graphe en largeur à partir d'un sommet (la source), consiste à l'explorer de la manière suivante, à partir d'un sommet particulier :

En partant de la source, on explore le graphe « en cercles concentriques » : on visite successivement tous les sommets à distance 1 de la source, puis tous les sommets à distance 2 de la source, et ainsi de suite jusqu'à ce que tous les sommets accessibles depuis la source aient été atteints.

Ex 2 : Sur le graphe orienté suivant G₁ en démarrant du sommet 'A' :



En partant de A, à distance 1 on trouve B et D, puis à distance 2 on trouve C et E, puis F à distance 3.

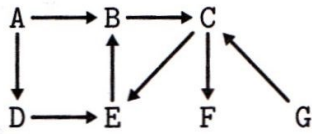
Compléter les commentaires et la construction du parcours sous forme de dictionnaire :

sommets à visiter sur le cercle courant	sommets à visiter sur le cercle suivant	commentaires	parcours : dictionnaire {sommets cochés :distance à la source}
A			
.	B,D	cocher A à distance 0,	{A :0}
. B,D	∅	s'éloigner de A	
. D	C	cocher B à distance 1	{A :0,B :1}
. . .	C,E	cocher D à distance 1	{A :0,B :1,D :1}
. . . C,E	∅	s'éloigner de A	
. . . E	F	cocher C à distance 2	
. . . .	F	cocher E à distance 2	
. . . . F	∅	s'éloigner de A	
∅	∅	cocher F à distance 3	
		Terminé	
sommets à distance d de la source	sommets à distance d+1 de la source		

Rq4. Comme pour le parcours en profondeur, il y a dans l'ordre des sommets parcourus sur chaque cercle une part d'indéterminisme.

Exo 10.

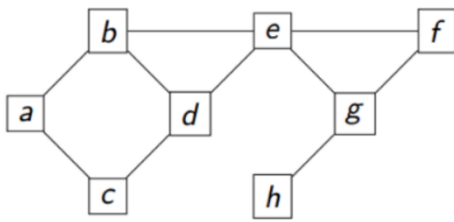
1. Donner la liste des sommets cochés dans un parcours en profondeur à partir de B



sommets à visiter sur le cercle courant (d)	sommets à visiter sur le cercle suivant (d+1)	commentaires	parcours : dictionnaire {sommets cochés :distance à la source}
B			
.	C	Cocher B	{B :0}
C	Vide	S'éloigner de B	
.	E,F	Cocher C	{B :0,C :1}
E,F	Vide	S'éloigner de C	
F	Vide	Cocher E	{B :0,C :1,E :2}
.		Cocher F	{B :0,C :1,E :2,F :2}
sommets à distance d de la source	sommets à distance d+1 de la source		

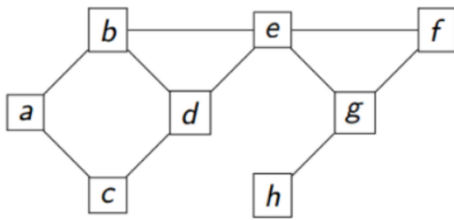
Rq5. Le sommet G n'a jamais été coché : il n'est pas accessible à partir de A, pas plus qu'à partir de n'importe quel autre sommet du graphe

Exo 11. Donner le dictionnaire des sommets cochés dans un parcours en largeur du graphe suivant non orienté en démarrant du sommet 'g' :



sommets à visiter sur le cercle courant (d)	sommets à visiter sur le cercle suivant (d+1)	commentaires	parcours : dictionnaire {sommets cochés :distance à la source}
g ∅	e,f,h	cocher g à distance 0	{g :0}

sommets à distance d de la source	sommets à distance d+1 de la source		
-----------------------------------	-------------------------------------	--	--



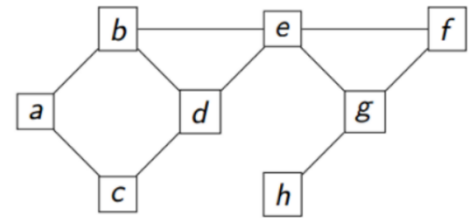
sommets à visiter sur le cercle courant (d)	sommets à visiter sur le cercle suivant (d+1)	commentaires	parcours : dictionnaire {sommets cochés :distance à la source}
g			
∅	e, f, h	cocher g à distance 0	{g :0}
e, f, h	∅	s'éloigner de g	
f,h	d,b	cocher e à distance 1	{g :0,e :1}
h	d,b	cocher f à distance 1	{g :0,e :1,f :1}
∅	d,b	cocher h à distance 1	{g :0,e :1,f :1,h :1}
d,b	∅	s'éloigner de g	
b	c	cocher d à distance 2	{g :0,e :1,f :1,h :1,d :2}
∅	c,a	cocher b à distance 2	{g :0,e :1,f :1,h :1,d :2,b :2}
c,a	∅	s'éloigner de g	
a	∅	cocher c à distance 3	{g :0,e :1,f :1,h :1,d :2,b :2,c :3}
∅	∅	cocher a à distance 3	{g :0,e :1,f :1,h :1,d :2,b :2,c :3,a :3}
sommets à distance d de la source	sommets à distance d+1 de la source		

4. IMPLEMENTATION

a) Version itérative



L'algorithme utilise deux files pour retenir les sommets à explorer sur chaque « cercle », mais l'ordre dans lequel figurent et sont retirés les sommets situés à une même distance de la source est indifférent. On pourrait aussi bien les mémoriser dans une pile ou mieux encore un ensemble (**set** en *Python*), dans lequel les éléments ne sont pas ordonnés.



fonction `parcours_largeur(G,source)`:

```
sommets_cochés ← {source : 0}
la_file_courante ← file vide
la_file_suivante ← file vide
On ajoute la source dans la_file_courante
d ← 0
Tant que la_file_courante n'est pas vide :
    s ← retirer de la_file_courante
    pour chaque voisin v de s
        si v n'est pas une clé dans le dictionnaire sommets_cochés
            on ajoute v : (d+1) au dictionnaire sommets_cochés
            on ajoute v dans la_file_suivante
    si la_file_courante est vide
        la_file_courante ← la_file_suivante
        la_file_suivante ← une autre file vide
        d ← d+1
renvoyer sommets_cochés
```

Exo 12. En utilisant une classe **File** et une classe **Graphe** de votre choix, implémenter cet algorithme de parcours en largeur. Tester sur les graphes **G_1** et **G_2**.

parcours_largeur(G_1,'A') renvoie : {'A': 0, 'B': 1, 'D': 1, 'C': 2, 'E': 2, 'F': 3}

parcours_largeur(G_2,'g') renvoie : {'g': 0, 'e': 1, 'f': 1, 'h': 1, 'b': 2, 'd': 2, 'a': 3, 'c': 3}



[..\SRC\parcours_largeur.py](#)

Exo 13. Un **plus court** chemin entre deux sommets

Le parcours en largeur permet comme le parcours en profondeur de trouver un chemin à partir d'un sommet origine. Le principe du parcours en largeur à partir de la source par cercles concentriques de rayons croissants permet de trouver **un plus court chemin** vers le sommet d'arrivée en nombre d'étapes (c'est-à-dire d'arêtes traversées).

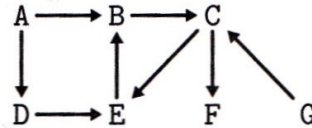
Comme pour la recherche d'un chemin en utilisant le parcours en profondeur, la méthode consiste à associer à chaque sommet dans un dictionnaire le sommet qui a permis de l'atteindre la première fois pendant le parcours en profondeur. (Cette information est perdue dans le cas d'une simple liste des sommets parcourus, elle est ici nécessaire)

Le sommet de départ ne sera associé à rien (**None**)

Une fois le parcours en largeur terminé, on cherche le sommet d'arrivée et on remonte le dictionnaire jusqu'au sommet de départ.

1. Ecrire une fonction semblable à la fonction de parcours en largeur, qui renvoie le dictionnaire des arcs parcourus où la clé est l'extrémité de l'arc et la valeur est l'origine de l'arc.

```
def parcours_arcs(g,depart):
    """renvoie le dictionnaire des arcs parcourus
    où la clé est l'extrémité de l'arc et la valeur est l'origine de l'arc"""
    # arcs_coches, dictionnaire à construire à partir du sommet depart,
    # est construit en lieu et place de la liste sommets_cochés
    # en parcourant le graphe en largeur
```



`parcours_largeur(G_1,'A')` renvoie {'A': 0, 'B': 1, 'D': 1, 'C': 2, 'E': 2, 'F': 3}
`parcours_arcs(G_1,'A')` renvoie {'A': None, 'B': 'A', 'D': 'A', 'C': 'B', 'E': 'D', 'F': 'C'}

2. Ecrire la fonction `un_chemin(g,depart,arrivee)` qui prend en paramètre le sommet de départ, le sommet d'arrivée, qui construit le dictionnaire des arcs d'un parcours en largeur, et qui renvoie le chemin sous forme de liste des sommets qui le constituent.
3. Tester cette fonction sur les graphes `G_1` et `G_2`. Le chemin produit sur les tests est-il toujours le plus court ?

b) Version récursive

```
def parcours_largeur_rec(g,source,sommets_coches=None,file_a_visiter=None):
```

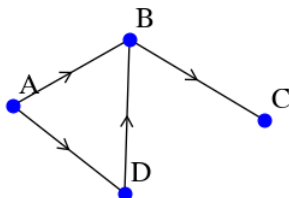


On peut utiliser un algorithme récursif pour parcourir un graphe en largeur.

1. on retire de la_file à visiter un sommet s (on l'affiche)
 2. pour chaque voisins non déjà visité de s
 3. on le note comme visité
 4. on l'ajoute à la_file
 5. on recommence à partir du 1
- Le processus s'arrête quand la_file est vide

Exo 14. En utilisant une classe **Graphe** au choix, implémenter cet algorithme de parcours en largeur. Tester sur les graphes `G_1` et `G_2`.

Exo 15. Dérouler à la main le parcours en largeur sur le graphe suivant, pour différentes valeurs du sommet de départ. Donner le contenu du dictionnaire renvoyé.



IV. Exos

Exo 16. Mystère

On donne l'algorithme suivant.

PARAMETRES

s : sommet (sommet quelconque)

G : un graphe

VARIABLES

u : sommet

v : sommet

p : pile (vide au départ)

pour tout sommet **u** du graphe **G**, **u.couleur** = blanc à l'origine

DEBUT

p.empiler(s)

tant que **p** n'est pas vide :

u ← **p.depiler()**

si **u** est noir :

 renvoie Vrai

sinon :

u.couleur ← noir

pour chaque sommet **v** adjacent au sommet **u** :

si **v.couleur** n'est pas noir :

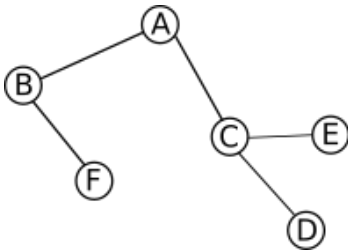
p.empiler(v)

fin tant que

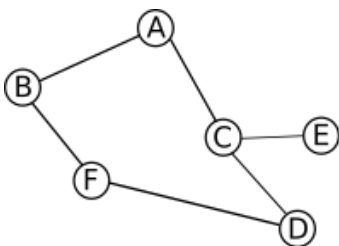
 renvoie Faux

FIN

1. Quel type de parcours utilise ce programme ?
2. Appliquer l'algorithme au graphe ci-dessous (à partir d'un sommet au choix).



3. Appliquer l'algorithme au graphe ci-dessous (à partir d'un sommet au choix).



4. Indiquer la signification des couleurs données aux sommets, donner un nom pertinent à la fonction *Python* qui implémente cet algorithme et rédiger sa *docstring*.

Exo 17. Chaines de mots

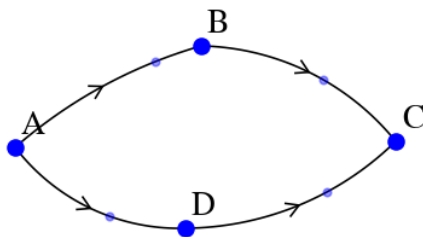
[TD Chaines de Mots\TD Chaines de Mots.docx](#)

Exo 18. Existence de cycles dans les graphes orientés (parcours tricolores)

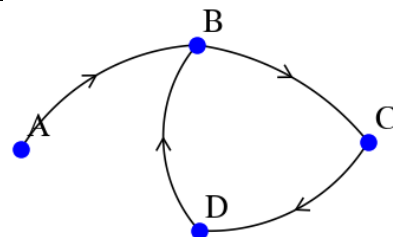
Les cycles sont plus compliqués à détecter dans le cas des graphes orientés, car la présence de plusieurs chemins entre deux sommets n'implique plus nécessairement la présence d'un cycle (c'est le cas dans un graphe non orienté).

On peut détecter les cycles à l'aide d'un simple parcours en profondeur, mais il nous faut pouvoir faire la différence entre les sommets que l'on rencontre plusieurs fois à cause d'un cycle et ceux que l'on rencontre plusieurs fois à cause de

la présence de plusieurs chemins.

GRAPHE G_4


le parcours à partir de A retombe sur C. Il n'y a pas de cycle pour autant.

GRAPHE G_5


Le parcours à partir de A retombe sur B. C'est bien à cause du cycle B-C-D.

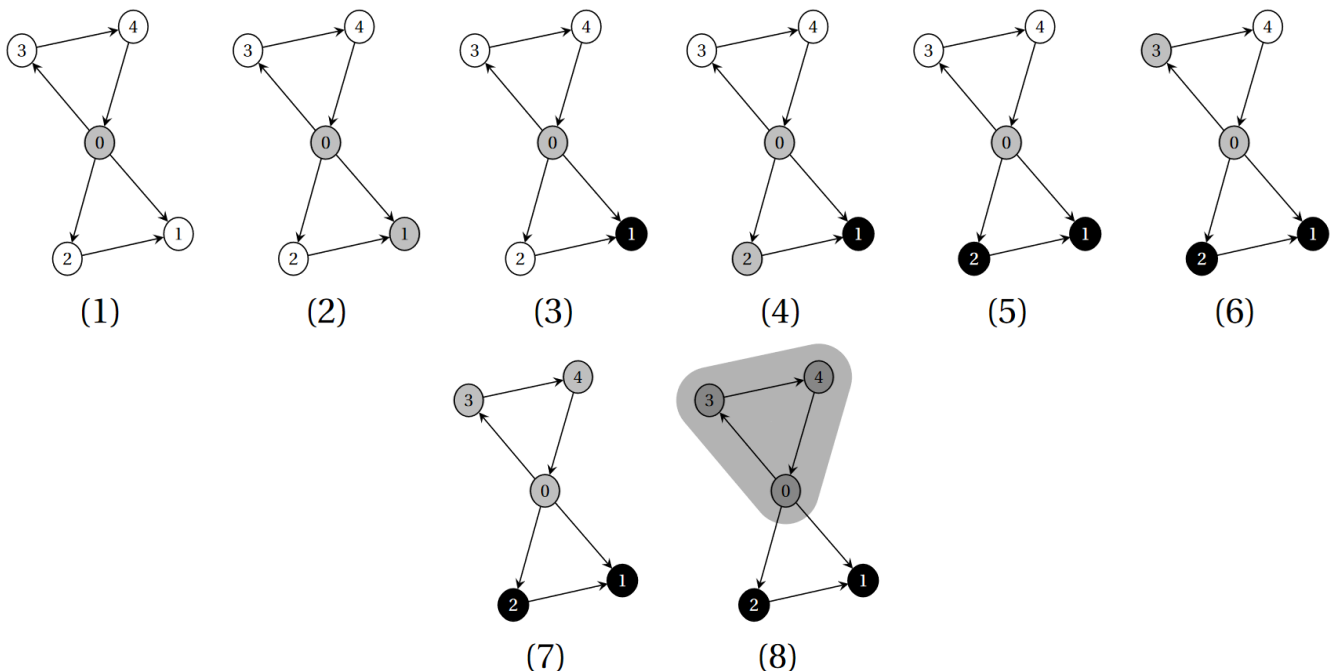
On étend le code de couleurs de la recherche de cycles dans un graphe non orienté :

Le parcours sera alors tricolore :

- ✚ "blanc" pour un sommet non encore visité ;
 - ✚ "gris" pour un sommet en cours de traitement, ce qui signifie que l'on est en train d'explorer ses successeurs ;
 - ✚ "noir" pour un sommet que l'on a fini de traiter, ce qui veut dire qu'on a également exploré tous ses successeurs.
- Ainsi, passer sur un sommet blanc signifie qu'on le rencontre pour la première fois, et passer sur un sommet noir signifie qu'on a trouvé un autre chemin pour y arriver sans passer par ses successeurs ; si l'on rencontre un sommet gris, cela signifie que l'on y est parvenu en passant par ses successeurs, et donc qu'on a identifié un cycle.



Pour exemple on donne le graphe G_6 suivant et les étapes de la détection de cycle suivant un parcours en profondeur tricolore au départ du sommet 0 :



Implémenter les deux fonctions suivantes :

```
def existe_parcours_o_cyclique(g,sommet):
    """detecte la présence d'un cycle dans le graphe orienté g par un parcours en
    profondeur à partir du sommet sommet """

def existe_cycle(g):
    """detecte la présence d'un cycle dans le graphe g
    renvoie True en presence d'un cycle, False sinon"""
```